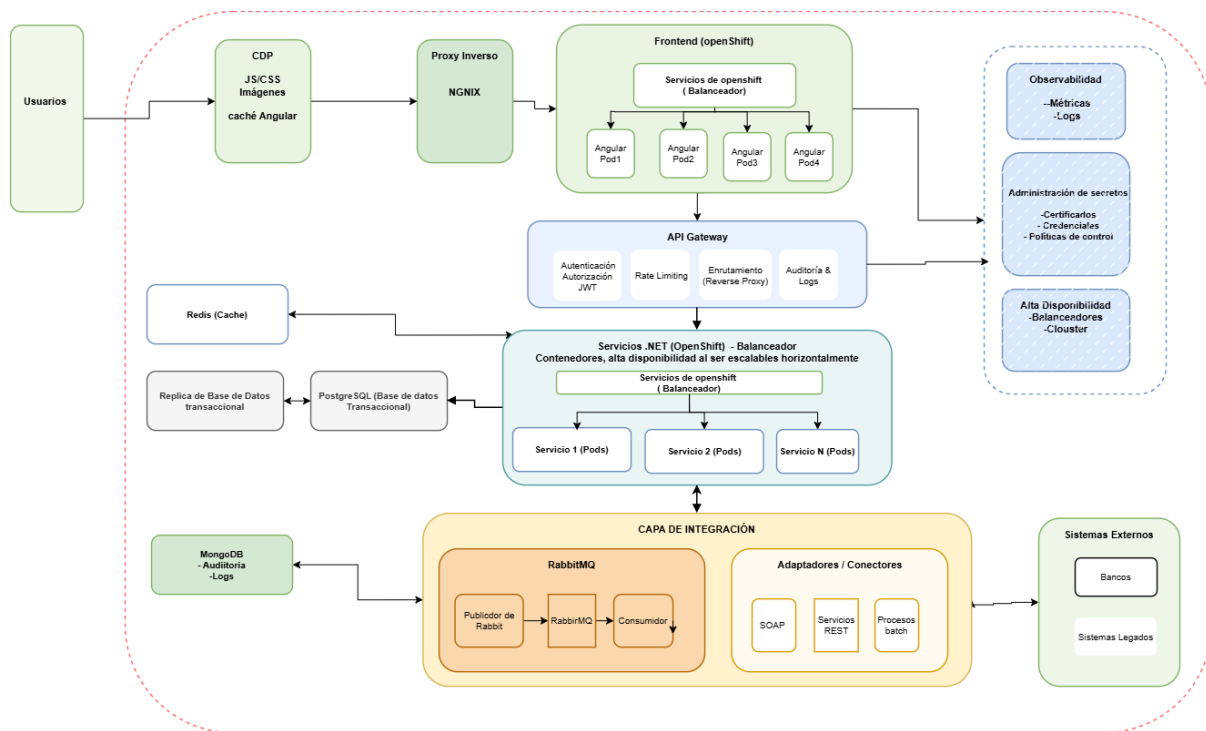


EXAMEN: Juan Luis Suchí Martínez

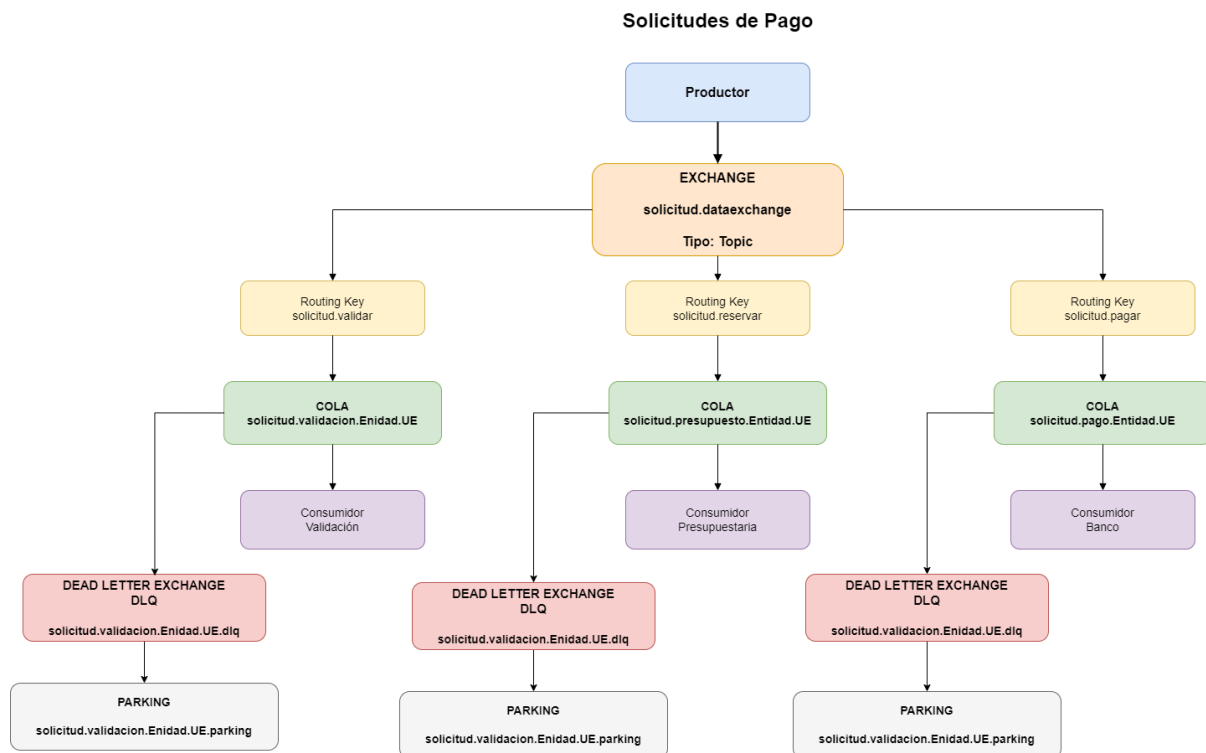
EXAMEN DE EXPERTO DE SISTEMAS FINANCIEROS

1. Arquitectura Objetivo

Se implementará una arquitectura que permita escalar horizontal y verticalmente los diferentes componentes de la solución, de acuerdo con las necesidades y la demanda del sistema. Esto permitirá distribuir la carga de trabajo, optimizar el uso de los recursos y garantizar la alta disponibilidad y continuidad del servicio, incluso ante incrementos de tráfico o fallos en alguno de sus componentes.

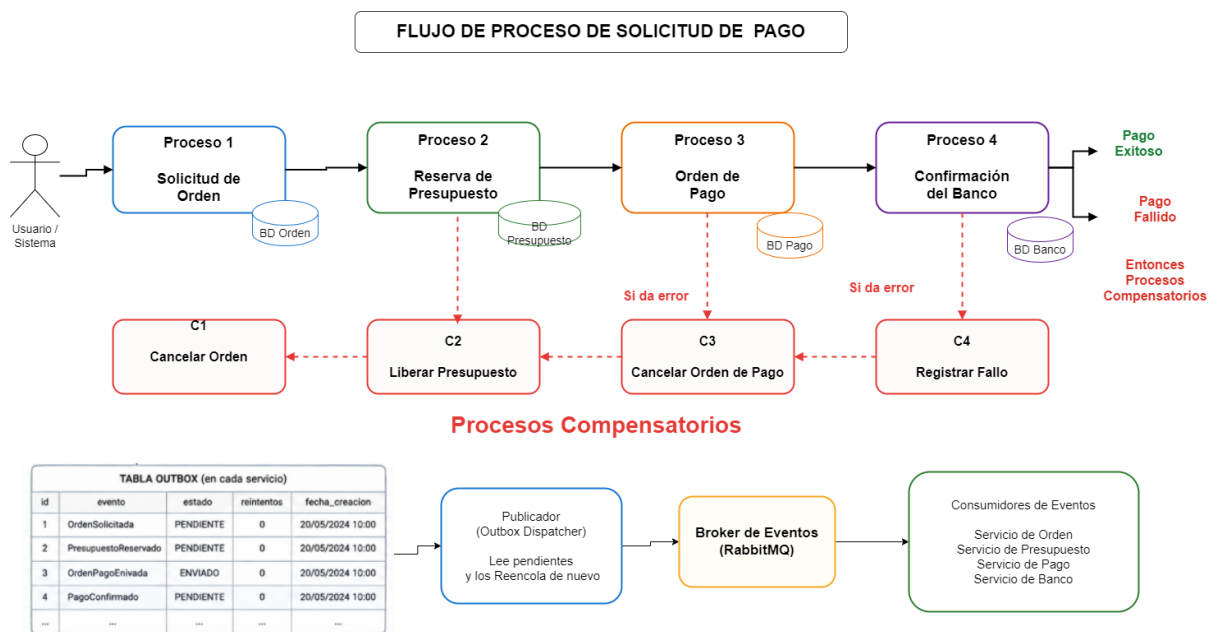


Base de Datos, Modelo relacional



- **Exchange:** Es el encargado de recibir los mensajes y enviarlos a la cola que corresponde.
- **Colas:** Es donde quedan guardados los mensajes mientras esperan ser procesados. Se utilizó el nombre solicitud seguido de la acción que realizará cada una.
- **Routing Key:** Es la llave que usa el Exchange para saber a qué cola tiene que mandar el mensaje.
- **Parking Lot:** Es donde se dejan los mensajes que ya fallaron varias veces para poder revisarlos después.
- **Acknowledgement:** Es la confirmación que indica que el mensaje se procesó correctamente. Si algo falla, no se confirma.
- **Prefetch:** Sirve para controlar cuántos mensajes puede tomar un consumidor al mismo tiempo y así evitar sobrecargarlo.
- **Idempotencia:** Sirve para evitar que un mismo mensaje se procese dos veces si por alguna razón llega repetido.
- **Orden:** Sirve para llevar una secuencia de los mensajes y evitar que se procesen en un orden incorrecto.
- **Correlación:** Es un identificador que permite darle seguimiento a una solicitud durante todo su recorrido.
- **Estrategia de reproceso:** Se revisarán los mensajes que llegaron a la DLQ y al solucionar el problema, se volverán a colocar en la cola para intentar procesarlos nuevamente.

3. Consistencia y procesos distribuidos



Para cada uno de los procesos que se tienen en el flujo se generan una serie de eventos, los cuales se van registrando en una tabla de outbox dentro de la Base de Datos. Esto permite llevar el control de los eventos que deben ser enviados a las diferentes colas y verificar si estos fueron o no procesados correctamente.

También la tabla nos permite identificar aquellos eventos que por algún error no pudieron ser consumidos, para luego un proceso los pueda verificar el estado en el que se encuentran e intentar nuevamente su envío hacia la cola que le corresponde, con ello nos evitamos la pérdida de mensajes.

Saga es un patrón que nos permite controlar una transacción que pasa por varios procesos con servicios diferentes, este nos permite seguir paso a paso cada una de ellas, si alguno de estos procesos falla, Saga permite ejecutar los procesos compensatorios para revertir por medio de otro proceso las acciones que se hayan realizado.

4. Desarrollo: Se subió el código, se realizó el backend y el Frontend

5. Seguridad

Autenticación: Se deben de mezclar varios tipos de autenticación

Algo que se, algo que poseo o una característica física, al poder mezclar una doble autenticación, me da más seguridad a los sistemas, además que las Apis se deberán de comunicar por JWT

Autorización: Es necesario manejar roles para el personal, de manera que cada usuario pueda acceder únicamente a las opciones o módulos que le corresponden dentro de la plataforma.

Almacenamiento de tokens: Los tokens se deben almacenar de forma segura y manejar un tiempo de expiración. También se debe evitar que puedan ser obtenidos fácilmente desde el navegador o por otros usuarios.

Protección de APIs: Las APIs deben estar protegidas para que solamente usuarios o sistemas autorizados puedan consumirlas. Se deben validar los tokens y permisos antes de permitir realizar una operación.

CORS: Se debe configurar CORS para indicar qué aplicaciones o dominios pueden consumir las APIs. De esta manera se evita permitir solicitudes desde sitios que no están autorizados.

CSP: Se debe utilizar CSP para controlar qué recursos puede cargar la aplicación, por ejemplo scripts, imágenes, permitir que sean solamente del sistema.

Secretos: Las contraseñas, cadenas de conexión y llaves no deben quedar directamente escritas dentro del código. Se deben guardar en variables de entorno o en un administrador de secretos, por ejemplo: Wallet de oracle o bien Secret de Openshit donde permite almacenar las credenciales y solamente utilizarlas con el nombre del secret.

Auditoría: Se debe guardar un registro de las operaciones importantes que realizan los usuarios, indicando quién realizó la acción, qué hizo y en qué fecha. Esto permite tener trazabilidad dentro del sistema.

Rate Limit: Se debe limitar la cantidad de peticiones que un usuario o aplicación puede realizar en determinado tiempo. Esto ayuda a evitar saturación de las APIs y posibles ataques a los servicios.

Controles para datos sensibles: Los datos sensibles deben manejarse de forma segura, encriptando al almacenarlos como al enviarlos. También se debe restringir su consulta únicamente a usuarios que realmente tengan autorización.

Es importante dentro de nuestras plataformas tener cada uno de lo que mencionamos a continuación, para poder mantener seguros los sistemas con la autenticación y la buena asignación de roles de usuario, además los cors es importante que los incluyamos en cada una de nuestras apis para que solo quien debe de tener permiso de conectarse lo pueda realizar.

El rate Limit también, es necesario hoy en día ya que si nos lanzan un set de peticiones en un cierto tiempo definido, se bloquea y deja de responder esto puede ayudar también por cualquier servicio que sea vulnerable y este retornando información que no debe, además de un buen monitoreo,

6. Estrategia de Entrega de Gobierno Técnico

- **SLIs/SLOs:** Son los indicadores y SLO los objetivos, nos sirven para medir si el sistema está funcionando bien y definir qué nivel de disponibilidad esperamos, por ejemplo, un 99.9% ahora un SLI por ejemplo, tiempo de respuesta, pagos exitosos, pagos pendientes, mensajes en cola, mensajes fallidos, mensajes reencolados, Tiempo de respuesta de los servicios, Tiempo de respuesta en la trazabilidad de un mensaje en una cola de algún proceso.
- **Logs estructurados:** Se guardará información de los procesos y errores para poder saber qué pasó cuando exista algún problema.
- **Métricas:** Se medirán tiempos de respuesta, cantidad de solicitudes, errores y uso de recursos, por Ejemplo, cantidad de mensajes pendientes de procesar.
- **Trazas distribuidas:** Permitirán seguir una solicitud durante todo su recorrido, desde que entra hasta que termina, aunque pase por varios servicios.
- **Alertas:** Se enviarán avisos cuando exista algún problema importante, como un servicio caído o demasiados errores. Es importante el monitoreo de los servidores para poder ver a tiempo si pasa alguna situación, o si es por falta de recursos o alguna falla de conectividad.
- **Dashboards:** Tendremos pantallas donde podamos visualizar rápidamente el estado y rendimiento del sistema, esto es muy útil porque nos permite reaccionar a tiempo por cualquier eventualidad.
- **Health Checks:** Cada servicio indicará si se encuentra funcionando correctamente y si puede conectarse a sus dependencias. Es importante también con la anterior de monitoreo, el poder llevarlas en conjunto para poder tener un tablero de como están respondiendo nuestras aplicaciones.
- **Capacidad:** Se monitoreará el consumo de CPU, memoria, almacenamiento y conexiones para evitar saturaciones.
- **Backup:** Se realizarán copias de seguridad de las bases de datos y de la información importante.

- **Recuperación:** Se tendrá definido cómo restaurar el sistema y la información en caso de una falla.
- **Pruebas de fallo:** Se harán pruebas simulando problemas, por ejemplo apagar un servicio o perder conexión con la base de datos, para comprobar que el sistema pueda recuperarse.

Considero que tener todos estos controles es importante porque nos permite saber realmente cómo está funcionando nuestro sistema, nos ayuda a detectar problemas a tiempo y poder reaccionar antes de que afecten a los usuarios o a los procesos. También permite identificar si una falla viene de la aplicación ó de los servidores, de una cola, de la base de datos.

Los backups y procesos de recuperación nos ayudan a estar preparados ante cualquier problema y evitar la pérdida de información. En conjunto, todo esto nos permite tener un sistema más estable, controlado y preparado para responder ante cualquier eventualidad.

7. Estrategia de entrega y gobierno técnico

- **Ramas:** Se trabajará con ramas separadas, por ejemplo
 - Main – Master
 - Capacitacion
 - QA
 - Desarrollo

para llevar un mejor control de los cambios antes de pasar a producción, y features las ramas que cada desarrollador realizara de Main o master, para poder hacer su desarrollo, las ramas dependen del giro del negocio de cada empresa y sus necesidades.

- **Revisión de código:** Antes de unir cambios importantes, otro desarrollador deberá revisar el código para detectar errores o mejoras.
Es importante que no solo una persona apruebe sus cambios, debe de haber de 1 a 2 personas involucradas en un pull request esto nos va a ayudar para poder controlar mejor el código, además de verificar las buenas o malas prácticas de los compañeros y hacérselas saber para que no pase en otras situaciones.
- **Análisis de seguridad:** Se revisará que el código no tenga vulnerabilidades, credenciales expuestas o configuraciones inseguras. Con aplicaciones externas que nos permita el poder verificar esta vulnerabilidad en nuestro código.

- **Pruebas automatizadas:** Se ejecutarán pruebas automáticamente para validar que los cambios nuevos no dañen funcionalidades que ya estaban trabajando.
- **CI/CD:** Se automatizará la compilación, pruebas y despliegue del sistema para reducir errores manuales.
- **Despliegue canario o Blue-Green:** Se utilizará una estrategia de despliegue controlado, liberando primero la nueva versión a una parte de los usuarios por esta razón selecciono el Despliegue canario, para poder ir validando cómo se comporta la nueva publicación.
- **Rollback:** Si una nueva versión presenta problemas, se tendrá la opción de regresar rápidamente a la versión anterior.
- **Documentación de arquitectura:** Se dejará documentado cómo está construido el sistema, sus servicios, integraciones, bases de datos y flujo de comunicación.
- **Proceso de aprobación:** Los cambios deberán pasar por desarrollo, pruebas y aprobación antes de ser publicados en producción.

Considero que implementar estas prácticas nos ayuda a tener un mejor control de los cambios que se realizan en el sistema. También permite detectar errores y problemas de seguridad antes de llegar a producción.

Las pruebas y los procesos de CI/CD ayudan a que los despliegues sean más rápidos y seguros. Tener estrategias de rollback nos permite regresar a una versión estable si algo sale mal. En conjunto, todo esto permite trabajar de una forma más ordenada, controlada y con menos riesgos.